МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра системного проектування

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ №3

з дисципліни «Професійний Пайтон»

на тему: «Ітератори, генератори та потокова обробка даних»

Виконав:
студент групи ФеП-32
Накладюк Володимир Тарасович
Варіант №15

Львів - 2026

1. Мета роботи

Сформувати практичні навички роботи з ітераційною моделлю Python: ітераторами, генераторами, `yield`, generator expressions, модулем itertools, принципом lazy evaluation та потоковою обробкою великих наборів даних без повного завантаження в пам’ять.

2. Індивідуальне завдання

Варіант 15. Потокова обробка результатів наукових експериментів. Вхідний формат: `experiment_id,name,researcher,parameter,result,date`. Необхідно реалізувати:

- streaming validation результатів експериментів;
- фільтрацію за researcher і фільтрацію result за заданим діапазоном;
- streaming average result та визначення minimum/maximum;
- pairwise-аналіз різниць між послідовними результатами;
- batch processing;
- отримання перших N experiment records і побудову lazy experimental pipeline.

3. Постановка задачі

До проєкту experiment-registry потрібно додати потоковий конвеєр, що складається з джерела даних і генераторів читання, очищення, parsing, validation, filtering, transformation та етапу агрегації. Конвеєр має використовувати generator expressions, щонайменше три функції з `yield`, конструкцію `yield from`, не менше трьох засобів itertools (зокрема `islice`), batch processing, обробляти не менше 100 000 записів. Потрібно порівняти eager і lazy варіанти за часом, пам’яттю й часом до першого результату, дослідити early termination.

4. Хід виконання

4.1. Вхідний набір даних

Кожен рядок CSV описує один експеримент: цілий ідентифікатор, назву, дослідника, вимірюваний параметр (temperature, pressure, voltage, frequency, concentration), числовий результат і дату у форматі ISO. Невеликий файл `data/experiments_sample.csv` містить навмисно некоректні рядки: нечисловий результат, порожній дослідник, неіснуюча дата, невідомий параметр і порожній рядок. Великі файли на 10 000 – 1 000 000 записів генерує `datagen.generate_test_file` (близько 1% рядків псуються випадковим чином).

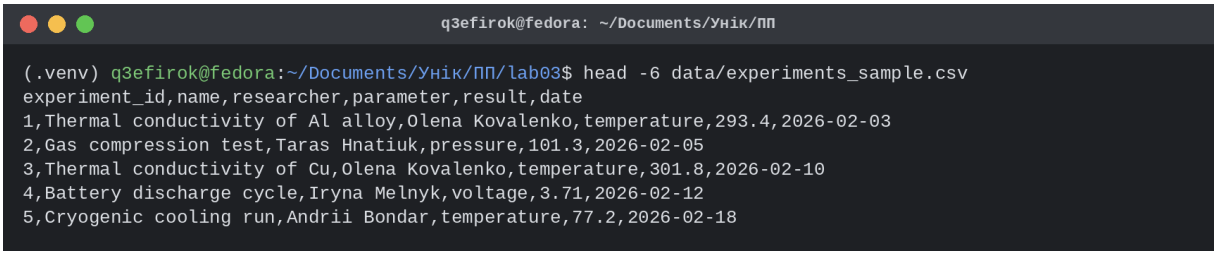


Рис. 3.1. Початок вхідного CSV-файлу

Лістинг 3.1. Генерація великого тестового файлу

```
def generate_test_file(path: Path, count: int, invalid_ratio: float = 0.01, seed: int =
15) -> Path:
    """Write `count` experiment rows; about invalid_ratio of them are broken on
purpose."""
    rng = random.Random(seed)
    start = date(2025, 1, 1)
    parameters = list(PARAMETERS)
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", encoding="utf-8", newline="") as file:
        writer = csv.writer(file)
        writer.writerow(CSV_HEADER)
        for experiment_id in range(1, count + 1):
            parameter = rng.choice(parameters)
            low, high = PARAMETERS[parameter]
            row = [
                str(experiment_id),
                f"{parameter.capitalize()} run {experiment_id}",
                rng.choice(RESEARCHERS),
                parameter,
                f"{rng.uniform(low, high):.3f}",
                (start + timedelta(days=rng.randrange(730))).isoformat(),
            ]
            if rng.random() < invalid_ratio:
                column, broken = rng.choice(INVALID_VALUES)
                row[column] = broken
            writer.writerow(row)
    return path
```

4.2. Структура проєкту

Потокову обробку винесено в окремий пакет `experiment_registry.streaming`, кожен модуль якого відповідає одному етапу конвеєра.

Лістинг 3.2. Структура проєкту lab03

```
lab03/
├── benchmarks/
│   ├── benchmark_search.py
│   └── benchmark_streaming.py
├── data/
│   └── experiments_sample.csv
├── src/
│   └── experiment_registry/
│       ├── commands/
│       │   ├── __init__.py
│       │   ├── analyze.py
│       │   ├── demo.py
│       │   └── stream.py
│       ├── streaming/
│       │   ├── __init__.py
│       │   ├── analytics.py
│       │   ├── datagen.py
│       │   ├── eager.py
│       │   ├── filters.py
│       │   ├── iterators.py
```

```
│       │   ├── parsers.py
│       │   ├── pipeline.py
│       │   └── readers.py
│       ├── __init__.py
│       ├── analytics.py
│       ├── config.py
│       ├── data.py
│       ├── decorators.py
│       ├── exceptions.py
│       ├── main.py
│       ├── models.py
│       ├── processors.py
│       └── services.py
├── .gitignore
├── pyproject.toml
└── README.md
```

4.3. Iterable та iterator

Iterable — об’єкт, який може повернути iterator через `__iter__()` (list, str, dict, файл). Iterator — об’єкт зі станом, що повертає наступний елемент через `__next__()` і сигналізує про завершення винятком `StopIteration`; його `__iter__()` повертає self. Цикл `for` викликає `iter()` один раз, а потім `next()` до `StopIteration`. Iterator можна пройти лише один раз, iterable — скільки завгодно, бо кожен прохід отримує новий iterator.

`ExperimentRecordIterator` реалізує протокол iterator безпосередньо й пропускає результати поза діапазоном. `ResultSeries` — це iterable з окремим класом iterator, тому його можна обходити повторно (на знімку другий прохід по iterator порожній, а `ResultSeries` використовується тричі).

Лістинг 3.3. Власні iterator і iterable

```
class ExperimentRecordIterator:
    """Iterator over experiments that skips results outside [low, high].

    Implements the iterator protocol directly: __iter__ returns self, __next__ returns
    the next suitable element or raises StopIteration. It can be consumed only once.
    """

    def __init__(self, experiments: Sequence[Experiment], low: float, high: float) -> None:
        self._experiments = experiments
        self._low = low
        self._high = high
        self._index = 0

    def __iter__(self) -> "ExperimentRecordIterator":
        return self

    def __next__(self) -> Experiment:
        while self._index < len(self._experiments):
            item = self._experiments[self._index]
            self._index += 1
            if self._low <= item.result <= self._high:
                return item
        raise StopIteration


class ResultSeries:
    """Iterable (not an iterator): every for-loop gets a fresh ResultSeriesIterator."""

    def __init__(self, results: Sequence[float]) -> None:
        self._results = list(results)

    def __iter__(self) -> "ResultSeriesIterator":
        return ResultSeriesIterator(self._results)
```

```
    def __len__(self) -> int:
        return len(self._results)


class ResultSeriesIterator:
    def __init__(self, results: list[float]) -> None:
        self._results = results
        self._position = 0

    def __iter__(self) -> "ResultSeriesIterator":
        return self

    def __next__(self) -> float:
        if self._position >= len(self._results):
            raise StopIteration
        value = self._results[self._position]
        self._position += 1
        return value
```

4.4. Generator functions, yield, yield from і generator expressions

Generator function містить `yield`: її виклик повертає generator object, а тіло виконується поступово, призупиняючись на кожному `yield` зі збереженням локального стану. `yield from` делегує генерацію іншому iterable — це використано в рекурсивному `flatten` і в `read_data_lines`. Нескінченний генератор `experiment_ids` безпечно обмежується `islice`; `list(count())` ніколи не завершився б.

Лістинг 3.4. Рекурсивний генератор і нескінченний потік

```
def flatten(values: Iterable[Any]) -> Iterator[Any]:
    """Recursive generator: flattens nested lists of measurement series via yield from."""
    for value in values:
        if isinstance(value, list):
            yield from flatten(value)
        else:
            yield value


def experiment_ids(start: int = 1) -> Iterator[str]:
    """Infinite generator of experiment codes: EXP-000001, EXP-000002, ..."""
    for number in count(start):
        yield f"EXP-{number:06d}"
```

Лістинг 3.5. Generator expression усередині yield from

```
def filter_by_researcher(experiments: Iterable[Experiment], researcher: str | None) ->
Iterator[Experiment]:
    """Keep experiments of one researcher (no-op when researcher is None)."""
    if researcher is None:
        yield from experiments
        return
    wanted = researcher.lower()
    yield from (item for item in experiments if item.researcher.lower() == wanted)
```

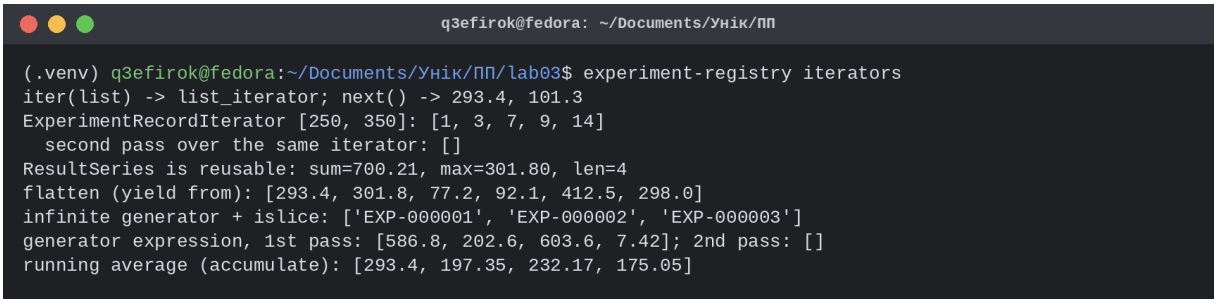


Рис. 3.2. Демонстрація iterator protocol і генераторів (підкоманда iterators)

Останні рядки знімка показують типову помилку: generator expression після першого проходу вичерпано, тому другий `list()` повертає порожній список.

4.5. Використані засоби itertools

Таблиця 3.1. Засоби itertools у проєкті

Засіб	Де використано	Призначення
`islice`	`take`, `batched`, демо нескінченного потоку	Перші N елементів, нарізання батчів
`pairwise`	`pairwise_differences`	Різниці між послідовними результатами
`chain.from_iterable`	`read_many`	Об’єднання кількох файлів в один потік
`accumulate`	`running_average`	Кумулятивне середнє
`groupby`	`count_consecutive_by_researcher`	Серії послідовних записів одного дослідника
`count`	`experiment_ids`	Нескінченний генератор кодів

4.6. Схема pipeline

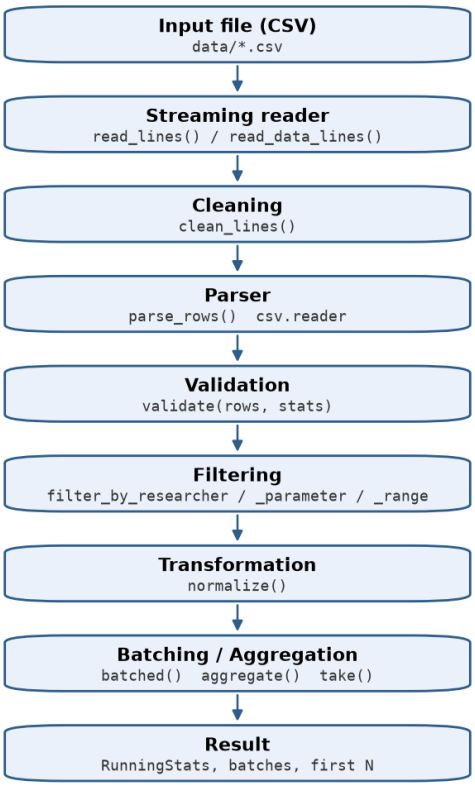


Рис. 3.3. Схема потокового конвеєра обробки експериментів

Лістинг 3.6. Побудова lazy pipeline

```
def build_pipeline(
    path: Path,
    researcher: str | None = None,
    parameter: str | None = None,
    low: float | None = None,
    high: float | None = None,
    stats: StreamStats | None = None,
) -> Iterator[Experiment]:
    """CSV -> read -> clean -> parse -> validate -> filter -> transform (all lazy)."""
    lines = read_data_lines(path)
    cleaned = clean_lines(lines)
    rows = parse_rows(cleaned)
    valid = validate(rows, stats)
    by_researcher = filter_by_researcher(valid, researcher)
    by_parameter = filter_by_parameter(by_researcher, parameter)
    in_range = filter_by_range(by_parameter, low, high)
    return normalize(in_range)
```

Функція `build_pipeline` лише з’єднує генератори й нічого не читає: перший рядок файлу буде прочитано тоді, коли споживач (aggregate, take, batched) попросить перший елемент.

4.7. Streaming reader

Читач відкриває файл у `with` і повертає рядки по одному, тож у пам’яті одночасно перебуває один рядок. Файл залишається відкритим, доки генератор не завершиться або не буде закритий; при достроковій зупинці `with` закриє файл під час `generator.close()` або збирання сміття.

Лістинг 3.7. Модуль readers.py

```
"""Streaming readers: one line at a time, the file is never loaded completely."""

from collections.abc import Iterator
from itertools import chain
from pathlib import Path


def read_lines(path: Path) -> Iterator[str]:
    """Yield lines of a text file lazily; the file stays open until iteration ends."""
    with path.open(encoding="utf-8", newline="") as file:
        for line in file:
            yield line


def read_data_lines(path: Path) -> Iterator[str]:
    """Yield lines without the CSV header."""
    lines = read_lines(path)
    next(lines, None)  # skip header
    yield from lines


def read_many(*paths: Path) -> Iterator[str]:
    """Concatenate several files into one stream with itertools.chain."""
    return chain.from_iterable(read_data_lines(path) for path in paths)
```

4.8. Етапи parsing і transformation

Лістинг 3.8. Очищення, parsing і transformation

```
def clean_lines(lines: Iterable[str]) -> Iterator[str]:
    """Drop blank lines and comments, strip trailing whitespace."""
    for line in lines:
        stripped = line.strip()
        if stripped and not stripped.startswith("#"):
            yield stripped


def parse_rows(lines: Iterable[str]) -> Iterator[RawRow]:
    """Parse CSV lines into dictionaries (csv.reader consumes the iterator lazily)."""
    for fields in csv.reader(lines):
        yield dict(zip(CSV_HEADER, (field.strip() for field in fields), strict=False))


def to_experiment(row: RawRow) -> Experiment:
    """Convert a raw row into an Experiment (raises ValueError on bad data)."""
    return Experiment(
        experiment_id=int(row["experiment_id"]),
        name=row["name"],
        researcher=row["researcher"],
        parameter=row["parameter"].lower(),
        result=float(row["result"]),
        date=date.fromisoformat(row["date"]),
    )


def normalize(experiments: Iterable[Experiment]) -> Iterator[Experiment]:
    """Transformation stage: normalize text fields and round results."""
    for item in experiments:
        item.researcher = " ".join(item.researcher.split()).title()
        item.result = round(item.result, 3)
        yield item
```

4.9. Validation і filtering

Validation перетворює рядок на `Experiment` (там перевіряються ID, назва, дослідник і діапазон результату), додатково перевіряє параметр і кількість полів.

Некоректні записи не зупиняють конвеєр: вони підраховуються в `StreamStats` разом із причиною.

Лістинг 3.9. Validation і filtering stage

```
@dataclass
class StreamStats:
    """Counters filled while the stream is being consumed."""

    read: int = 0
    valid: int = 0
    invalid: int = 0
    reasons: Counter[str] = field(default_factory=Counter)


def validate(rows: Iterable[RawRow], stats: StreamStats | None = None) ->
Iterator[Experiment]:
    """Streaming validation: yield valid experiments, count and skip invalid rows."""
    stats = stats if stats is not None else StreamStats()
    for row in rows:
        stats.read += 1
        try:
            if len(row) != 6:
                raise ExperimentValidationError("wrong number of fields")
            experiment = to_experiment(row)
            if experiment.parameter not in PARAMETERS:
                raise ExperimentValidationError(f"unknown parameter {experiment.parameter!
r}")
        except (ValueError, KeyError, ExperimentValidationError) as error:
            stats.invalid += 1
            stats.reasons[type(error).__name__] += 1
            continue
        stats.valid += 1
        yield experiment


def filter_by_range(experiments: Iterable[Experiment], low: float | None,
                    high: float | None) -> Iterator[Experiment]:
    """Keep results inside [low, high]; open bounds are allowed."""
    lower = float("-inf") if low is None else low
    upper = float("inf") if high is None else high
    for item in experiments:
        if lower <= item.result <= upper:
            yield item
```

4.10. Aggregation і batching

`RunningStats` накопичує кількість, суму, мінімум і максимум за один прохід, не зберігаючи значень, тому потребує O(1) пам’яті. `batched` нарізає потік на списки фіксованого розміру через `islice`; `find_first` реалізує lazy search із early termination.

Лістинг 3.10. Aggregation stage і pairwise

```
@dataclass
class RunningStats:
    """Streaming average / minimum / maximum (no list of values is stored)."""

    count: int = 0
    total: float = 0.0
    minimum: Experiment | None = None
    maximum: Experiment | None = None

    def add(self, item: Experiment) -> None:
        self.count += 1
        self.total += item.result
        if self.minimum is None or item.result < self.minimum.result:
            self.minimum = item
        if self.maximum is None or item.result > self.maximum.result:
```

```
            self.maximum = item

    @property
    def average(self) -> float | None:
        return self.total / self.count if self.count else None


def pairwise_differences(experiments: Iterable[Experiment]) -> Iterator[tuple[int, int, float]]:
    """Difference between consecutive results: (previous_id, current_id, delta)."""
    for previous, current in pairwise(experiments):
        yield previous.experiment_id, current.experiment_id, round(current.result -
previous.result, 3)
```

Лістинг 3.11. Перші N записів, batching, early termination

```
def take(items: Iterable[T], n: int) -> list[T]:
    """First n elements of a (possibly infinite) stream via islice."""
    return list(islice(items, n))


def batched(items: Iterable[T], size: int) -> Iterator[list[T]]:
    """Split a stream into lists of `size` elements (the last batch may be shorter)."""
    if size < 1:
        raise ValueError("batch size must be >= 1")
    iterator = iter(items)
    while batch := list(islice(iterator, size)):
        yield batch


def find_first(items: Iterable[T], predicate: Callable[[T], bool]) -> T | None:
    """Lazy search with early termination: stops at the first match."""
    for item in items:
        if predicate(item):
            return item
    return None
```

4.11. Результати роботи конвеєра

```
q3efirok@fedora: ~/Documents/Унік/ПП

(.venv) q3efirok@fedora:~/Documents/Унік/ПП/lab03$ experiment-registry stream
data/experiments_sample.csv --parameter temperature --range 250 350 --first 4 --batch 2
Input: data/experiments_sample.csv
Read: 16  valid: 12  invalid: 4  reasons: {'ValueError': 2, 'ExperimentValidationError': 2}
Filter: researcher=None, parameter=temperature, range=[250.0, 350.0]
Matched: 4  streaming average: 310.535
Minimum: 293.400 (#1)  maximum: 348.900 (#7)

First 4 records (islice):
  #1       Olena Kovalenko  temperature     293.400  2026-02-03
  #3       Olena Kovalenko  temperature     301.800  2026-02-10
  #7       Olena Kovalenko  temperature     348.900  2026-03-02
  #15      Maria Shevchuk   temperature     298.040  2026-03-24

Pairwise differences of consecutive results:
  #1 -> #3: +8.400
  #3 -> #7: +47.100
  #7 -> #15: -50.860

Runs of the same researcher (groupby) in the first 12 valid rows:
  [('Olena Kovalenko', 1), ('Taras Hnatiuk', 1), ('Olena Kovalenko', 1), ('Iryna Melnyk', 1), ('Andrii
Bondar', 1), ('Taras Hnatiuk', 1), ('Olena Kovalenko', 1), ('Iryna Melnyk', 1), ('Andrii Bondar', 1),
('Olena Kovalenko', 1), ('Maria Shevchuk', 2)]

Batch processing (batch size 2):
  batch 1: 2 records, average 297.600
  batch 2: 2 records, average 323.470
  total batches: 2

Early termination: first result >= 346.50 is #7 (348.900)
```

Рис. 3.4. Обробка тестового файлу з некоректними записами

Із 16 рядків 4 відкинуто validation, а «maria shevchuk» нормалізовано до «Maria Shevchuk». Далі конвеєр запущено на 100 000 записів.

```
q3efirok@fedora: ~/Documents/Унік/ПП

(.venv) q3efirok@fedora:~/Documents/Унік/ПП/lab03$ experiment-registry generate
data/generated/experiments_100000.csv --count 100000
Generated 100000 records -> data/generated/experiments_100000.csv (7.1 MB)
(.venv) q3efirok@fedora:~/Documents/Унік/ПП/lab03$ experiment-registry stream
data/generated/experiments_100000.csv --researcher "Olena Kovalenko" --range 100 300 --first 5 --batch
5000
Input: data/generated/experiments_100000.csv
Read: 100000  valid: 99031  invalid: 969  reasons: {'ValueError': 380, 'ExperimentValidationError': 589}
Filter: researcher=Olena Kovalenko, parameter=None, range=[100.0, 300.0]
Matched: 3100  streaming average: 200.000
Minimum: 100.118 (#80014)  maximum: 299.837 (#15575)

First 5 records (islice):
  #1       Olena Kovalenko  pressure        156.634  2025-02-07
  #45      Olena Kovalenko  pressure        135.850  2026-02-19
  #46      Olena Kovalenko  pressure        200.868  2025-01-04
  #97      Olena Kovalenko  temperature     115.569  2025-04-07
  #113     Olena Kovalenko  pressure        221.750  2026-08-25

Pairwise differences of consecutive results:
  #1 -> #45: -20.784
  #45 -> #46: +65.018
  #46 -> #97: -85.299
  #97 -> #113: +106.181

Runs of the same researcher (groupby) in the first 12 valid rows:
  [('Olena Kovalenko', 2), ('Andrii Bondar', 1), ('Sofiia Romaniuk', 1), ('Petro Lysenko', 1), ('Sofiia
Romaniuk', 1), ('Taras Hnatiuk', 1), ('Dmytro Savchuk', 1), ('Andrii Bondar', 1), ('Maria Shevchuk', 1),
 ('Taras Hnatiuk', 1), ('Maria Shevchuk', 1)]

Batch processing (batch size 5000):
  batch 1: 3100 records, average 200.000
  total batches: 1

Early termination: first result >= 297.00 is #19 (2506.544)
```

Рис. 3.5. Генерація та потокова обробка 100 000 записів

4.12. Eager і lazy implementation

Eager-версія читає весь файл `readlines()`, створює повний список рядків, потім список словників і список експериментів, і лише потім фільтрує. Lazy-версія передає кожен запис через усі етапи по одному.

Лістинг 3.12. Eager implementation

```
def load_valid_experiments(path: Path) -> list[Experiment]:
    """Read the whole file, then parse, then validate - three full lists in memory."""
    with path.open(encoding="utf-8", newline="") as file:
        lines = file.readlines()[1:]
    rows = [dict(zip(CSV_HEADER, fields, strict=False)) for fields in csv.reader(lines)]
    experiments = []
    for row in rows:
        try:
            item = to_experiment(row)
        except (ValueError, KeyError, ExperimentValidationError):
            continue
        if item.parameter in PARAMETERS:
            experiments.append(item)
    return experiments


def eager_filter(path: Path, low: float, high: float) -> list[Experiment]:
    return [item for item in load_valid_experiments(path) if low <= item.result <= high]
```

Лістинг 3.13. Порівнювані функції та вимірювання peak memory

```
def eager_average(path: Path) -> tuple[int, float]:
    items = eager_filter(path, LOW, HIGH)
    return len(items), sum(item.result for item in items) / len(items)


def lazy_average(path: Path) -> tuple[int, float]:
    stats = aggregate(build_pipeline(path, low=LOW, high=HIGH))
    return stats.count, stats.average or 0.0


def peak_memory(func: Callable[[Path], Any], path: Path) -> float:
    """Peak traced memory in MB; the pipeline is fully consumed inside func."""
    tracemalloc.start()
    func(path)
    _, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    return peak / 1024 / 1024
```

5. Експериментальна частина

Для кожного набору (10 000, 100 000, 500 000 записів) обчислювалися кількість і середнє результатів у діапазоні [100, 300]. Час — найкращий із трьох запусків `perf_counter`; peak memory — через `tracemalloc` в окремому запуску з повним споживанням pipeline; time to first result — час до отримання першого відфільтрованого запису. Збіг результатів eager і lazy перевірено `assert`.

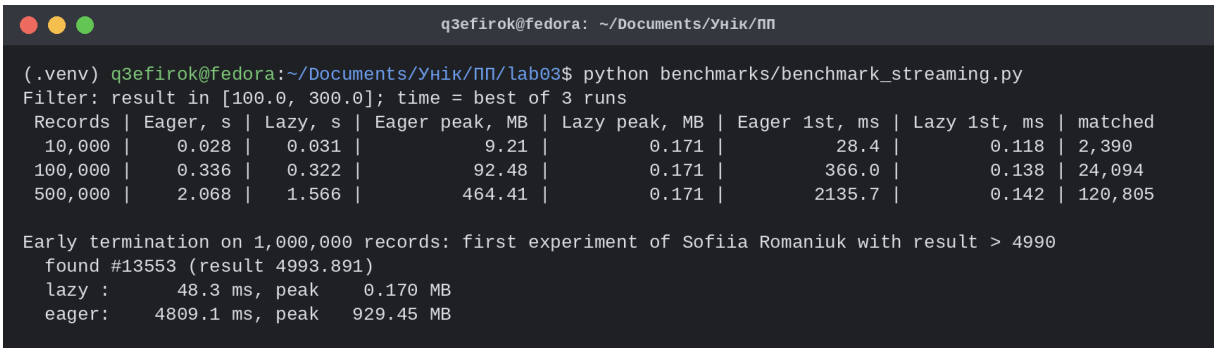


Рис. 3.6. Результати benchmark eager vs lazy

Таблиця 3.2. Elapsed time

Records	Eager time, с	Lazy time, с	Matched records
10 000	0.028	0.031	2 390
100 000	0.336	0.322	24 094
500 000	2.068	1.566	120 805

Таблиця 3.3. Peak memory і time to first result

Records	Eager peak memory, МБ	Lazy peak memory, МБ	Eager 1st result, мс	Lazy 1st result, мс
10 000	9.21	0.171	28.4	0.118
100 000	92.48	0.171	366.0	0.138
500 000	464.41	0.171	2135.7	0.142

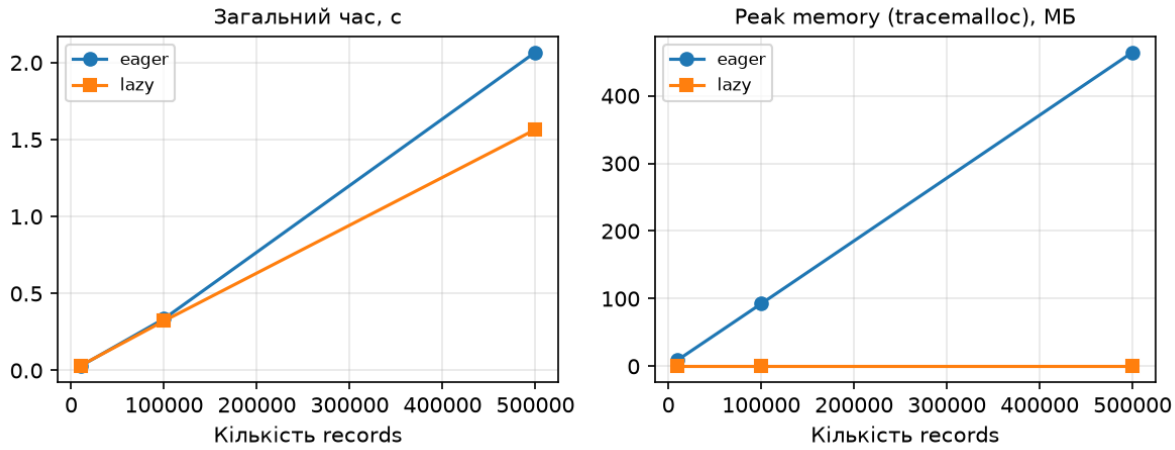


Рис. 3.7. Час виконання та peak memory eager і lazy варіантів

Таблиця 3.4. Early termination на 1 000 000 записів

Варіант	Час, мс	Peak memory, МБ
Lazy (find_first)	48.3	0.170
Eager (повний список)	4809.1	929.45

5.1. Аналіз результатів

Пам’ять eager-варіанта зростає пропорційно кількості записів (з 9.21 до 464.41 МБ), бо всі проміжні списки існують одночасно. Lazy-варіант використовує сталі ≈0.171 МБ незалежно від розміру файлу. Загальний час обох варіантів близький (2.068 с проти 1.566 с на 500 000 записів): обидва виконують ту саму роботу з parsing і validation, а невеликий виграш lazy-версії пояснюється меншим навантаженням на алокатор. Отже, генератор не є «завжди швидшим» — його головна перевага в пам’яті.

Time to first result відрізняється на порядки: lazy-конвеєр повертає перший запис за частки мілісекунди, тоді як eager спершу обробляє весь файл (2135.7 мс). Ще помітніше це в задачі early termination: lazy-пошук зупинився після кількох тисяч рядків (48.3 мс, 0.170 МБ), а eager прочитав мільйон записів (4809.1 мс, 929.45 МБ). Недоліки lazy evaluation: генератор можна пройти лише один раз, помилки з’являються під час споживання, а не створення, і для повторного використання дані доводиться читати знову або матеріалізувати.

5.2. Завдання підвищеної складності

- власні iterator classes і рекурсивний генератор `flatten` із `yield from`;
- infinite generator + `islice`; `chain`, `accumulate`, `pairwise`, `groupby`;
- pipeline із декількох файлів (`read_many`) і підрахунок invalid records з причинами;
- вимірювання time-to-first-result і lazy search з early termination;
- порівняння пам’яті для list і generator через tracemalloc.

6. Відповіді на контрольні запитання

1. Що таке iterable?

Iterable — це об'єкт, з якого можна отримати iterator за допомогою iter(); для цього він реалізує метод __iter__() (або старий протокол __getitem__ з індексами від 0). Iterable є list, tuple, str, dict, set, range, file object, generator. Його можна передати в цикл for, sum(), list(), sorted(), а контейнери на кшталт list можна обходити багаторазово, бо кожен iter() дає новий iterator.

2. Що таке iterator?

Iterator — об'єкт, який зберігає поточний стан обходу й видає елементи по одному: метод __next__() повертає наступний елемент, а після вичерпання піднімає StopIteration. Також він реалізує __iter__(), що повертає self, тому сам є iterable. Прикладами є результат iter([1, 2, 3]), file object, generator object і мій власний iterator class для записів експериментів.

3. Чим iterable відрізняється від iterator?

Iterable лише вміє створити iterator через __iter__() і сам стану обходу не зберігає, тому list можна обходити багато разів: кожен iter() дає новий незалежний iterator. Iterator має __next__(), пам'ятає поточну позицію та є одноразовим. Кожен iterator є iterable, але не навпаки: для list вираз iter(values) is values дає False, а next(values) викликає TypeError.

4. Для чого використовується iter()?

Вбудована функція iter(obj) отримує iterator з iterable, викликаючи obj.__iter__(); якщо об'єкт не iterable, виникає TypeError. Для iterator вона повертає той самий об'єкт. Форма iter(callable, sentinel) створює iterator, що викликає функцію, доки та не поверне sentinel. У pipeline iter() використовують, наприклад, щоб batching через islice послідовно брав елементи з одного спільного iterator.

5. Для чого використовується next()?

next(iterator) викликає iterator.__next__() і повертає наступний елемент, просуваючи стан обходу на один крок; після вичерпання піднімається StopIteration. Форма next(iterator, default) замість винятку повертає default. Наприклад, next(gen, None) зручно використати, щоб отримати перший запис експерименту, який відповідає умові, без обробки решти файла.

6. Що робить __iter__()?

__iter__() — спеціальний метод, який викликають функція iter() і цикл for; він повинен повертати iterator. У контейнері (iterable) він щоразу створює новий об'єкт-iterator, а в самому iterator повертає self. Якщо __iter__() реалізувати як generator function з yield, кожен його виклик автоматично повертатиме новий незалежний generator.

7. Що робить __next__()?

__next__() — метод iterator, що повертає наступний елемент і оновлює внутрішній стан (поточну позицію); коли елементів більше немає, він піднімає StopIteration. Його викликають функція next() і цикл for. У моєму власному iterator class для

експериментів __next__() повертає черговий запис або сигналізує про кінець даних через raise StopIteration.

8. Що таке StopIteration?

StopIteration — вбудований виняток, яким iterator у __next__() сигналізує, що елементи закінчилися. Цикл for, list(), sum() та інші споживачі перехоплюють його автоматично й нормально завершують обхід. У generator його не піднімають вручну: достатньо return або дійти до кінця функції, а явний StopIteration усередині generator з Python 3.7 перетворюється на RuntimeError (PEP 479).

9. Як цикл for використовує iterator protocol?

Цикл for item in values спочатку викликає iter(values), щоб отримати iterator, потім на кожній ітерації викликає next(iterator) і присвоює результат змінній item. Коли next() піднімає StopIteration, цикл перехоплює виняток і завершується (виконується гілка else, якщо вона є). Концептуально це еквівалент while True з try: item = next(iterator) except StopIteration: break.

10. Чому iterator часто називають one-shot object?

Iterator зберігає лише поточну позицію й рухається тільки вперед, без можливості повернутися на початок. Після вичерпання кожен наступний next() знову піднімає StopIteration, тому повторний обхід нічого не дає: list(it) вперше поверне [1, 2, 3], а вдруге — []. Щоб пройти дані ще раз, потрібно створити новий iterator з початкового iterable або заново викликати generator function.

11. Чому iter(iterator) is iterator?

За протоколом метод __iter__() в iterator повертає self, тобто iterator сам є своїм iterator. Це потрібно, щоб iterator можна було передавати туди, де очікується iterable (цикл for, list(), функції itertools), без створення копії. Наслідок: усі посилання вказують на один і той самий стан обходу, тому споживання через одне з них просуває й інші.

12. Чи можна повторно використати exhausted generator?

Ні. Вичерпаний generator object перебуває в завершеному стані: кожен наступний next() піднімає StopIteration, а list(gen) повертає []. Перезапустити його неможливо, потрібно створити новий generator, повторно викликавши generator function (наприклад, заново збудувати pipeline від read_lines), або, якщо даних небагато, один раз матеріалізувати їх у list.

13. Що таке generator function?

Generator function — це функція, що містить хоча б один yield; її виклик не виконує тіло, а повертає generator object, який реалізує iterator protocol. Тіло виконується поступово при кожному next(), видаючи значення через yield. У моєму pipeline read_lines, parse_rows, validate та етапи фільтрації є generator functions, кожна з яких обробляє записи CSV по одному.

14. Яку роль виконує yield?

yield повертає чергове значення споживачеві й призупиняє виконання generator, зберігаючи локальні змінні та поточну позицію в коді. При наступному next()

виконання продовжується з інструкції після yield. Наприклад, validate через yield видає один коректний запис експерименту, і наступний рядок CSV читається лише тоді, коли споживач попросить наступний елемент.

15. Чим yield відрізняється від return?

return завершує функцію остаточно й повертає одне значення, після чого локальний стан втрачається. yield лише призупиняє generator, видаючи одне з потенційно багатьох значень, а стан зберігається до наступного next(). Усередині generator return завершує ітерацію (спричиняє StopIteration), а значення з return value доступне лише як StopIteration.value або як результат виразу yield from.

16. Коли фактично починає виконуватися generator function?

Виклик generator function лише створює generator object, не виконуючи жодного рядка тіла. Код починає виконуватися при першому next() (або першій ітерації for, sum(), list()) і працює до першого yield. Тому, наприклад, read_lines(path) не відкриває файл одразу, а помилка FileNotFoundError виникне лише під час споживання pipeline.

17. Що зберігає generator між викликами next()?

Generator зберігає свій frame: значення всіх локальних змінних і аргументів, позицію призупинення (інструкцію після останнього yield), стан внутрішніх циклів і блоків try/with, а також посилання на вкладені iterators. Наприклад, read_lines пам'ятає відкритий file object і позицію в ньому, а parse_rows — посилання на попередній stage та поточний стан розбору CSV.

18. Що таке generator expression?

Generator expression — вираз у круглих дужках із синтаксисом comprehension, наприклад (e.result for e in records if e.researcher == name), який створює generator object, що обчислює елементи ліниво, по одному за запитом. Якщо це єдиний аргумент функції, зайві дужки можна опустити: sum(e.result for e in records). Він зручний для простих етапів mapping і filtering без окремої функції.

19. Чим generator expression відрізняється від list comprehension?

List comprehension [...] одразу (eager) обчислює всі елементи й зберігає їх у list, займаючи пам'ять пропорційно кількості елементів, зате дозволяє індексування, len() і багаторазовий обхід. Generator expression (...) створює lazy generator зі сталим обсягом пам'яті, видає елементи по одному, є одноразовим і не підтримує індексацію та len(). Для мільйона записів експериментів різниця в peak memory дуже суттєва.

20. Що таке lazy evaluation?

Lazy evaluation (ліниве обчислення) — стратегія, за якої значення обчислюються лише тоді, коли вони справді потрібні споживачеві, по одному елементу. У Python її реалізують generators, generator expressions, map, filter, zip, range та itertools. Наприклад, мій pipeline над CSV нічого не читає, доки етап агрегації не почне запитувати записи, і в пам'яті одночасно перебуває лише поточний рядок.

21. Що таке eager evaluation?

Eager evaluation (енергійне обчислення) — стратегія, за якої всі значення обчислюються одразу й повністю матеріалізуються в структурі даних, наприклад через list comprehension, list(), sorted() чи file.read(). Результат готовий до індексування й повторного використання, але потребує пам'яті під усю колекцію. В eager-варіанті моєї роботи всі записи CSV спочатку завантажуються в list і лише потім обробляються.

22. Які переваги lazy evaluation?

Lazy evaluation дає менший peak memory, практично незалежний від розміру dataset, можливість обробляти файли, більші за RAM, і нескінченні послідовності, швидший time to first result та early termination, коли обробка зупиняється після першого потрібного результату. Lazy stages також легко комбінувати в pipeline. Саме різницю в peak memory показує порівняння eager і lazy варіантів через tracemalloc.

23. Які недоліки lazy evaluation?

Lazy-дані одноразові: generator не можна повторно обійти, індексувати чи дізнатися його len() без матеріалізації. Помилки проявляються із затримкою під час споживання, що ускладнює налагодження; ресурси, як-от відкриті файли, утримуються довше; накладні витрати на виклики generators можуть зробити загальний час більшим, ніж в eager-варіанті. Крім того, сортування чи медіана все одно потребують усіх даних.

24. Що таке streaming?

Streaming — обробка даних послідовно, невеликими частинами (рядок, запис або batch) у міру надходження, без повного завантаження всього набору в пам'ять. Дані проходять крізь етапи обробки, а результати агрегуються інкрементально. У моїй роботі CSV з експериментами читається порядково, кожен запис проходить parsing, validation і фільтрацію та враховується в streaming average і min/max.

25. Чому streaming важливий для великих files?

Файл розміром у гігабайти може не поміститися в RAM, а streaming тримає в пам'яті лише поточний рядок або batch, тож peak memory майже не залежить від розміру файла. Обробка починається одразу, без очікування повного завантаження, і її можна зупинити достроково. Тому мій pipeline однаково працює і на 100 тисячах, і на мільйоні записів експериментів.

26. Чому read() може бути проблемою для multi-GB file?

file.read() без аргументу читає весь вміст файла в один об'єкт str (або bytes) в оперативній пам'яті. Для файла в кілька GB це потребує щонайменше стільки ж RAM, а з декодуванням UTF-8 і подальшим split() — у кілька разів більше, що може спричинити MemoryError, свопінг або аварійне завершення процесу. Натомість файл варто ітерувати порядково або читати блоками фіксованого розміру через read(size).

27. Чому file object є iterable?

Текстовий file object реалізує iterator protocol: його __iter__() повертає сам об'єкт, а __next__() з використанням внутрішнього буфера читає й повертає наступний рядок разом із символом нового рядка. Тому for line in file читає файл порядково без

завантаження цілком. Фактично file object є iterator, тож повторно обійти його можна лише після повернення позиції через seek(0).

28. Для чого використовується yield from?

yield from iterable делегує генерацію іншому iterable або sub-generator: усі його елементи передаються споживачеві так, ніби їх видавав поточний generator, що замінює цикл for item in it: yield item. Також він прозоро передає send() і throw() та повертає значення return вкладеного generator. Наприклад, read_lines може видавати рядки через yield from file, а рекурсивний generator — через yield from для вкладених структур.

29. Що таке itertools?

itertools — модуль стандартної бібліотеки з ефективними, реалізованими на C будівельними блоками для iterators: нескінченними (count, cycle, repeat), такими, що обробляють послідовності (islice, chain, takewhile, dropwhile, accumulate, pairwise, groupby, batched), і комбінаторними (product, permutations, combinations). Майже всі вони lazy. У моєму pipeline використано islice, pairwise, batched, chain, accumulate і groupby.

30. Для чого itertools.count()?

count(start=0, step=1) створює нескінченний lazy iterator арифметичної прогресії start, start + step, start + 2·step і так далі; step може бути дробовим. Його використовують для генерації ідентифікаторів, нумерації елементів потоку разом із zip() або як джерело нескінченного потоку. Наприклад, count(1) може генерувати послідовні experiment_id під час створення тестового CSV.

31. Чому count() потрібно обмежувати?

count() ніколи не піднімає StopIteration, тому будь-яке повне споживання (list(), sum(), sorted(), for без break) ніколи не завершиться, а функції, що матеріалізують дані, необмежено споживатимуть пам'ять. Обмежувати потік потрібно через islice(count(), n), takewhile() з умовою зупинки, zip() зі скінченним iterable або явний break у циклі.

32. Для чого islice()?

islice(iterable, stop) або islice(iterable, start, stop, step) ліниво бере зріз будь-якого iterator без його повної матеріалізації, на відміну від звичайного slicing, який працює лише з послідовностями; від'ємні індекси не підтримуються. У моєму pipeline islice використовується, щоб отримати перші N experiment records без обробки всього файла, обмежити нескінченні потоки та сформувати batches.

33. Для чого chain()?

chain(*iterables) ліниво об'єднує декілька iterables в один послідовний потік: спочатку видає всі елементи першого, потім другого і так далі, не створюючи спільного list. Варіант chain.from_iterable() приймає один iterable, що містить джерела. У моєму проєкті chain дозволяє обробляти записи експериментів із кількох CSV-файлів як єдиний потік, що подається на вхід pipeline.

34. Як працює takewhile()?

takewhile(predicate, iterable) видає елементи, доки predicate повертає True, і остаточно зупиняється на першому елементі, для якого умова False; цей елемент уже спожито з вихідного iterator, але не повернуто. Наприклад, takewhile(lambda x: x < 5, [1, 2, 3, 10, 2]) дасть [1, 2, 3]. Він корисний для early termination на впорядкованих даних і обмеження нескінченних потоків.

35. Як працює dropwhile()?

dropwhile(predicate, iterable) пропускає початкові елементи, поки predicate повертає True, а починаючи з першого елемента, для якого умова False, видає його та всі подальші елементи вже без перевірки умови. Наприклад, dropwhile(lambda x: x < 5, [1, 2, 3, 10, 2]) дасть [10, 2]. Це зручно, щоб пропустити службові рядки чи «розігрів» на початку потоку.

36. Для чого accumulate()?

accumulate(iterable, func, *, initial=None) ліниво видає проміжні накопичені результати; за замовчуванням це сума, тож для [10, 20, 30] отримаємо 10, 30, 60. Замість додавання можна передати іншу бінарну функцію, наприклад max для поточного максимуму. У моєму pipeline accumulate дає running sum або поточний максимум результатів експериментів, не зберігаючи весь потік.

37. Для чого pairwise()?

pairwise(iterable) (Python 3.10+) ліниво повертає пари сусідніх елементів, що перекриваються: (a, b), (b, c), (c, d) і так далі; з n елементів виходить n − 1 пара. Це зручно для часових рядів, обчислення різниць і змін показників. У моєму варіанті pairwise використовується для аналізу різниць між послідовними результатами експериментів: current.result − previous.result.

38. Як працює groupby()?

groupby(iterable, key=None) ліниво проходить потік і об'єднує в групу послідовні елементи з однаковим значенням key, повертаючи пари (key, group_iterator); нова група починається щоразу, коли key змінюється. Group iterator використовує спільний базовий iterator, тому його треба спожити до переходу до наступної групи. Наприклад, після сортування за researcher можна порахувати кількість і середній result кожного дослідника.

39. Чому перед groupby() часто потрібне sorting?

groupby() групує лише сусідні елементи з однаковим key, на відміну від SQL GROUP BY. Якщо записи одного researcher розкидані по файлу, той самий ключ з'явиться кількома окремими групами. Тому потік спочатку сортують за тим самим key, але sorted() матеріалізує всі дані й знищує memory advantage, тож для великих потоків замість цього часто використовують агрегацію через dict або Counter.

40. Що робить product()?

product(*iterables, repeat=1) ліниво генерує декартовий добуток: усі кортежі, у яких по одному елементу взято з кожного iterable, що еквівалентно вкладеним циклам for. Наприклад, product(["A", "B"], [1, 2, 3]) дає шість пар від ("A", 1) до ("B", 3).

Кількість результатів дорівнює добутку довжин вхідних iterables, як у сітці комбінацій параметрів експерименту.

41. Чим combinations() відрізняється від permutations()?

combinations(iterable, r) генерує вибірки довжини r без урахування порядку й без повторень, тому (A, B) і (B, A) вважаються однією комбінацією; їх C(n, r) = n!/(r!(n − r)!). permutations(iterable, r) враховує порядок, тож (A, B) і (B, A) є різними результатами; їх n!/(n − r)!. Для ["A", "B", "C"] при r = 2 це 3 комбінації та 6 перестановок.

42. Чому combinatorial iterators можуть бути небезпечними навіть при lazy execution?

Lazy execution економить лише пам'ять: елементи генеруються по одному, але їхня кількість зростає факторіально або експоненційно, наприклад 20! ≈ 2,4·10^18 перестановок. Повний перебір триватиме практично нескінченно, а list() з таких результатів вичерпає пам'ять. Крім того, product, permutations і combinations спершу зчитують вхідні iterables у кортежі, тож їх треба обмежувати через islice або фільтри.

43. Що таке batching?

Batching — групування елементів потоку в пакети (batches) фіксованого розміру для подальшої обробки цілим пакетом. У Python 3.12+ це робить itertools.batched(iterable, n), що ліниво видає кортежі по n елементів (останній може бути коротшим), а в ранніших версіях batching реалізують через islice. У моєму pipeline записи експериментів обробляються batches, і в пам'яті одночасно перебуває лише поточний batch.

44. Які переваги batching?

Batching зменшує накладні витрати на кожну операцію: один запит до database, API чи запис у файл обробляє одразу багато записів, що значно швидше за поелементну обробку. Водночас пам'ять обмежена розміром batch, а не всього dataset. Також batches зручні для векторних обчислень, транзакцій, логування прогресу, повторних спроб після помилок і паралельної обробки.

45. Що таке lazy pipeline?

Lazy pipeline — ланцюжок lazy stages (generators, generator expressions, функції itertools), де кожен етап приймає iterator попереднього й повертає новий iterator, а дані проходять конвеєр по одному елементу лише тоді, коли їх запитує кінцевий споживач. Проміжні collections не створюються. У мене це read_lines → parse_rows → validate → фільтри за researcher і діапазоном result → transformation → streaming aggregation.

46. Які stages може містити pipeline?

Типовий pipeline містить source (читання файла чи генерацію даних), очищення (strip, пропуск порожніх рядків), parsing (перетворення рядків у записи), validation (відкидання некоректних записів), filtering, transformation (mapping, нормалізацію), batching, обмеження через islice і sink — aggregation, запис у файл або виведення. У моєму проєкті це read_lines, parse_rows, validate, фільтри, transform і aggregate.

47. Що є source у streaming pipeline?

Source — початковий етап, який породжує потік даних і з якого наступні етапи беруть елементи. Це може бути файл, що читається порядково, мережевий потік, курсор database, API, generator синтетичних даних або нескінченний iterator на кшталт count(). У моїй роботі source — generator read_lines, який через with open(...) і yield порядково читає великий CSV з експериментами.

48. Що є transformation?

Transformation — проміжний етап, який перетворює кожен елемент потоку на інший, зберігаючи поштучну обробку: parsing рядка в запис, приведення типів, нормалізацію, обчислення похідних полів (mapping). Реалізується generator function з yield, generator expression або map(). У моєму pipeline transform-етап отримує валідні записи експериментів і, наприклад, нормалізує поля або формує потрібне для аналізу представлення.

49. Що є sink?

Sink — кінцевий етап pipeline, який споживає потік і саме цим запускає всі попередні lazy stages, адже generators самі нічого не обчислюють. Прикладами є sum(), цикл for, list(), запис у файл чи database, streaming aggregation. У моєму проєкті sink — етап aggregate, що за один прохід обчислює кількість записів, streaming average і minimum/maximum result.

50. Чи гарантує generator менший total execution time?

Hi. Generator гарантує насамперед менший peak memory і швидший time to first result, але не загальну швидкість: кожен next() і перемикання frame мають накладні витрати, тож на даних, що поміщаються в пам'яті, eager-варіант (list comprehension, вбудовані функції над list) може бути не повільнішим або навіть швидшим. Lazy виграє в часі за early termination або коли eager спричиняє свопінг.

51. Яка основна memory advantage generator?

Generator зберігає лише свій стан (frame) і поточний елемент, а не всю послідовність, тому займає сталий обсяг пам'яті, O(1), незалежно від кількості елементів. Сам generator object має розмір порядку сотень байтів, тоді як list на мільйон записів займає десятки чи сотні мегабайт. Це підтверджує вимірювання peak memory через tracemalloc під час порівняння eager і lazy варіантів.

52. Що таке time to first result?

Time to first result — час від запуску обробки до отримання першого готового результату. В eager-підході він включає завантаження й обробку всього dataset, а в lazy pipeline перший запис одразу проходить усі stages, тож результат з'являється майже миттєво. Цю метрику вимірюють, наприклад, через time.perf_counter() від старту до отримання першого елемента для eager і lazy варіантів.

53. Чому lazy pipeline добре підтримує early termination?

У lazy pipeline кожен наступний елемент обчислюється лише на запит споживача, тож коли споживач зупиняється (break, next() для першого збігу, islice, takewhile), решта даних не читається й не обробляється взагалі. Наприклад, пошук першого

експерименту певного researcher у файлі з мільйона записів може завершитися вже після кількох рядків, тоді як eager-варіант спочатку будує весь відфільтрований list.

54. Чому exception у generator може виникати із затримкою?

Код generator function виконується не під час виклику, а лише при споживанні через next(). Тому FileNotFoundError у read_lines, ValueError під час перетворення result на float чи KeyError з'являться не в місці створення pipeline, а пізніше — там, де sink почне його споживати, можливо, на мільйонному рядку. Це ускладнює діагностику, тому очікувані помилки варто обробляти всередині відповідного stage, наприклад у validate.

55. Чому list(generator) може бути проблемою?

list(generator) повністю споживає generator і матеріалізує всі елементи в пам'яті, тобто знищує memory advantage lazy-підходу: для мільйона записів експериментів це сотні мегабайт. Після цього generator вичерпаний, а для нескінченного generator такий виклик ніколи не завершиться. Тому list() доречний лише для невеликих або обмежених через islice потоків, наприклад у тестах.

56. Що трапиться при list(count())?

count() — нескінченний iterator, тому list() безкінечно викликатиме next() і додаватиме числа до списку, ніколи не отримавши StopIteration. Програма зависне, а споживання пам'яті зростатиме, доки не виникне MemoryError або процес не буде примусово завершено операційною системою (OOM killer). Правильно спершу обмежити потік: list(islice(count(), 10)).

57. Як generator може впливати на lifecycle file resource?

Якщо generator відкриває файл у блоці with, файл залишається відкритим, доки generator не дійде до кінця, а не лише під час створення чи одного виклику. Якщо споживання припинено достроково (break, islice), файл утримується, доки generator не закриють через gen.close() або збирач сміття, що піднімає GeneratorExit у точці yield і виконує вихід із with. Тому generator варто закривати явно, наприклад через contextlib.closing.

58. Як тестувати невеликий generator?

Невеликий скінченний generator тестують, передаючи йому підготовлені вхідні дані (наприклад, list рядків CSV замість файла) і матеріалізуючи результат через list(), який порівнюють з очікуваним у pytest через assert. Окремо перевіряють поетапну видачу через next(), кінець потоку через pytest.raises(StopIteration), лінивість (тіло не виконується до першого next()) і граничні випадки: порожній вхід та некоректні записи для validate.

59. Як тестувати infinite generator?

Infinite generator ніколи не передають цілком у list() чи sum(); у тесті беруть обмежену кількість елементів через islice, наприклад assert list(islice(gen, 5)) == [1, 2, 3, 4, 5], або кілька разів поспіль викликають next(). Також можна перевірити очікуваний префікс через takewhile чи zip зі скінченним iterable і те, що стан коректно зберігається між викликами next().

60. Коли краще використовувати list, а коли generator?

list доречний, коли даних небагато, потрібні індексування, len(), сортування чи повторний обхід, або результат використовується кілька разів. Generator кращий для великих чи нескінченних потоків, однопрохідної обробки, pipeline зі streaming aggregation та early termination. Тому для CSV з мільйоном експериментів я використовую generators, а невеликий результат, наприклад перші N записів, можна матеріалізувати в list.

7. Висновок

У лабораторній роботі до проєкту experiment-registry додано пакет потокової обробки, у якому дані проходять через ланцюжок генераторів: читання, очищення, parsing, validation, фільтрація, нормалізація, batching і агрегація. Реалізовано власні iterator і iterable, генератори з `yield` і `yield from`, generator expressions і шість засобів itertools, а також усі вимоги варіанта 15: streaming validation, фільтри за дослідником і діапазоном, потокові середнє/мінімум/максимум, pairwise-різниці, batch processing і перші N записів.

Експеримент показав, що lazy-конвеєр тримає сталий обсяг пам’яті (≈0.171 МБ) на будь-якому розмірі файлу, тоді як eager-варіант потребує 464.41 МБ на 500 000 записів. За загальним часом варіанти близькі, але lazy дає перший результат майже миттєво і дозволяє early termination, що на 1 000 000 записів скоротило час пошуку з 4809.1 до 48.3 мс. Lazy evaluation доцільна для великих і нескінченних потоків, а eager — коли дані невеликі й потрібні повторно.